

Họ và tên:	Bộ môn CNPM
MSSV: STT:	

ĐỀ THI MÔN THIẾT KẾ VÀ XÂY DỰNG PHẦN MỀM (Kỳ 20241)

(Thời gian: 100 phút – Không sử dụng tài liệu, chép bài hoặc cho chép bài sẽ nhận điểm 0).

Câu 1. (1.0 điểm – Điểm thưởng) Liệt kê tối đa 02 điều em thích ở môn học này, và tối thiểu 03 điều cần cải thiện.

Câu 2. Bệnh viện Z cần phát triển một hệ thống Terلودermatology hỗ trợ chẩn đoán các bệnh lý về da cho các bác sĩ chăm sóc ban đầu tại tuyến y tế cơ sở. Hệ thống sử dụng một dịch vụ trí tuệ nhân tạo chẩn đoán qua hình ảnh (*DermAI*) để tạo ra các đánh giá sơ bộ. Sau đó bệnh nhân có thể được bác sĩ chuyên khoa tư vấn chi tiết.

Bệnh nhân (*Patient*) mới phải tạo tài khoản bằng cách cung cấp thông tin cá nhân của họ, bao gồm các thông tin như tên (*name*), email (*email*), số điện thoại (*phone number*) và mật khẩu (*password*). Bệnh nhân đã đăng ký có thể đăng nhập bằng email và mật khẩu của họ, đồng thời tính năng quản lý mật khẩu cho phép bệnh nhân đặt lại mật khẩu thông qua xác minh email.

Bệnh nhân có thể xem lại hồ sơ y tế của mình (*patient records*) kèm theo các thông tin về các lần chẩn đoán và các giao dịch thanh toán đã thực hiện. Để tạo yêu cầu chẩn đoán, bệnh nhân cần đăng nhập vào hệ thống và hoàn thành hồ sơ y tế cơ bản.

Trước khi yêu cầu chẩn đoán chi tiết (*detailed-diagnosis request*), bệnh nhân cần yêu cầu chẩn đoán sơ bộ (*pre-diagnosis request*) bằng cách tải lên hình ảnh da tổn thương của họ và nhận kết quả qua *DermAI*. Mỗi tài khoản sẽ có 02 lần gửi yêu cầu miễn phí. Các yêu cầu tiếp theo bệnh nhân cần thanh toán qua các cổng thanh toán phổ biến (ví dụ: PayPal, thẻ tín dụng/ghi nợ).

Khi yêu cầu chẩn đoán sơ bộ, bệnh nhân có thể lựa chọn một trong 02 loại chẩn đoán: tổn thương da (*skin lesion*) và phân tích nốt ruồi (*mole analysis*) với giá thành (*price*) và thông tin chẩn đoán chi tiết (*detailed diagnosis*) khác nhau. Sau đó:

- Bệnh nhân tải lên một hình ảnh vùng tổn thương da.
- Hệ thống trước hết sẽ kiểm tra thông tin ảnh gồm kiểu tệp (jpg hoặc png) và kích thước tệp phù hợp (không quá 2MB).
- Hệ thống tiếp theo kiểm tra chất lượng hình ảnh (độ phân giải, độ sáng, độ nét, góc chụp) đạt yêu cầu. Nếu hình ảnh không đáp ứng thì hệ thống thông báo để bệnh nhân thay đổi hình ảnh khác.
- Tiếp theo, bệnh nhân nhập thông tin bổ sung: vị trí tổn thương (*location on body*), thời gian xuất hiện (*duration*), triệu chứng đi kèm (*symptoms*), tình trạng trước đó (*previous conditions*).
- Hình ảnh và các thông tin này sẽ được gửi đến hệ thống *DermAI* để phân tích kết quả sơ bộ (*pre-diagnosis result*).
- Kết quả sẽ bao gồm các khả năng bệnh lý có thể xảy ra (*diagnosis*), xác suất cho mỗi khả năng (*probability*), mức độ nghiêm trọng (*severity*), và đề xuất các bước tiếp theo (*recommendation*).
- Tùy theo dịch vụ bệnh nhân đã chọn trước đó, hệ thống sẽ gửi thêm các thông tin tùy theo loại dịch vụ như sau:
 - o Skin lesion result: màu sắc (*lesion color*), hình dạng (*lesion shape*) tổn thương và bệnh lý (*pathology*).
 - o Mole analysis result: chỉ số ABCDE của nốt ruồi (*mole index*), khả năng ác tính.
- Hệ thống lưu kết quả vào hồ sơ y tế và gửi thông báo kết quả đến email của bệnh nhân.
- Trong tình huống dịch vụ *DermAI* bị lỗi thì hệ thống sẽ ghi log lỗi và thực hiện hoàn lại thanh toán cho bệnh nhân.
- Ngoài ra hệ thống cung cấp hình thức thông báo kết quả qua tin nhắn SMS và thông báo đẩy (Push Notification) theo thiết lập hồ sơ của bệnh nhân.

Các bác sĩ được cung cấp tài khoản để thực hiện các nhiệm vụ chuyên môn. Khi một hồ sơ bệnh nhân có một kết quả chẩn đoán từ *DermAI* và bệnh nhân đó có gửi yêu cầu chẩn đoán chi tiết, bác sĩ sẽ kiểm tra và bổ sung các chẩn đoán chi tiết. Sau khi đã có thông tin chẩn đoán chi tiết, bệnh nhân sẽ được thông báo về kết quả chi tiết giống như hình thức thông báo chẩn đoán sơ bộ.

Các thao tác quản lý hệ thống được thực hiện thông qua bảng quản trị (admin panel). Quản trị viên (admin) có thể quản lý tài khoản bệnh nhân và bác sỹ, bao gồm xem, chỉnh sửa thông tin và khóa tài khoản.

Hệ thống cũng cung cấp các thiết lập kết nối đến DermAI, các báo cáo về lỗi và thống kê sử dụng dịch vụ. Các dữ liệu về hình ảnh và hồ sơ y tế của bệnh nhân được truyền tải và lưu trữ an toàn theo quy định y tế.

- a. **(1.0 điểm)** Hãy vẽ biểu đồ use case bao gồm các use case liên quan đến bệnh nhân. Lưu ý vẽ đầy đủ các tác nhân cần thiết không chỉ bao gồm con người.
- b. **(2.5 điểm)** Cho mã nguồn sau đây minh họa xử lý gửi thông báo cho người dùng theo nhiều cách khác nhau. Đánh giá chất lượng của thiết kế. Đây có phải một thiết kế tốt (ví dụ: low coupling, high cohesion, tuân theo nguyên tắc SOLID)? Giải thích vì sao. Cung cấp giải pháp và mã nguồn chỉnh sửa của em cho bất kỳ vấn đề nào được xác định.

```
public class UserNotification {
    private List<String> emailRecipients = new ArrayList<>();
    private List<String> smsRecipients = new ArrayList<>();
    public void sendNotification(User user, String message, NotificationType type) {
        switch (type) {
            case 0: //EMAIL
                sendEmail(user.getEmail(), message);
                break;
            case 1: //SMS
                sendSMS(user.getPhoneNumber(), message);
                break;
            case 2: //PUSH
                sendPushNotification(user.getDeviceToken(), message);
                break;
            default:
                throw new IllegalArgumentException("Unsupported notification type");
        }
    }
    private void sendEmail(String email, String message) {
        // Logic gửi email
    }
    private void sendSMS(String phoneNumber, String message) {
        // Logic gửi SMS
        try {
            // Giả định sử dụng Twillio
            String accountSid = "YOUR_ACCOUNT_SID";
            String authTokens = "YOUR_AUTH_TOKEN";

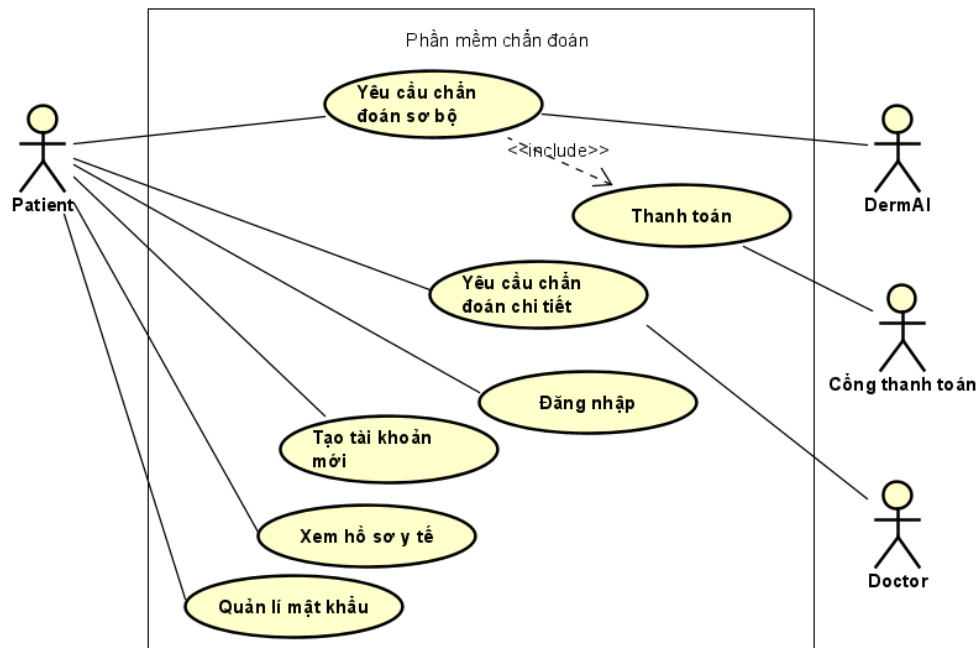
            // Xử lý gửi SMS với Twillio
        } catch (Exception e) {
            System.err.println("SMS sending failed: " + e.getMessage());
        }
    }
    private void sendPushNotification(String deviceToken, String message) {
        // Logic gửi push notification
    }
}
```

- c. **(5.0 điểm)** Hãy thực hiện các yêu cầu sau cho use case “Yêu cầu chẩn đoán sơ bộ (Make pre-diagnosis request)”, bao gồm:
 - Thiết kế biểu đồ lớp chi tiết (mức thiết kế). Để đơn giản hóa, em không cần thiết kế phần **thanh toán**, coi đó là một use case riêng biệt.
 - Viết mã nguồn cho các lớp chính đủ để minh họa ý tưởng thiết kế của mình.
 - Thiết kế các trường hợp kiểm thử cho use case trên.
- d. **(1.5 điểm)** Trong tương lai, hệ thống có thể gặp một số yêu cầu phát sinh hoặc thay đổi như phía dưới đây (đây là những yêu cầu gợi ý nhưng có thể không phải tất cả). Hãy phân tích và đánh giá thiết kế của em. Nếu thiết kế không tốt, em cần chỉ ra rõ ràng và đưa ra giải pháp chung cho những vấn đề đó. Các mẫu thiết kế (design patterns) có thể được sử dụng để giải quyết các vấn đề. Tuy nhiên, trong mọi tình huống, em cần giải thích tại sao với mỗi giải pháp, thiết kế lại được cải thiện và giải quyết được vấn đề.
 - Các thông tin có thể thay đổi, thêm hoặc bớt như các loại bệnh lý chẩn đoán, loại triệu chứng...
 - Hình thức gửi thông báo có thể bổ sung thêm một kênh mới như gọi điện tự động, Zalo...
 - Có thể có nhiều loại dịch vụ chẩn đoán hơn, ví dụ: phát ban, vết chàm. Mỗi loại có thông tin chi tiết khác nhau.

ĐÁP ÁN THAM KHẢO

Lưu ý: Đáp án chỉ mang tính chất tham khảo, có thể còn nhiều sai sót.

Câu (a): Lưu ý chỉ vẽ các UC và tác nhân liên quan đến bệnh nhân.

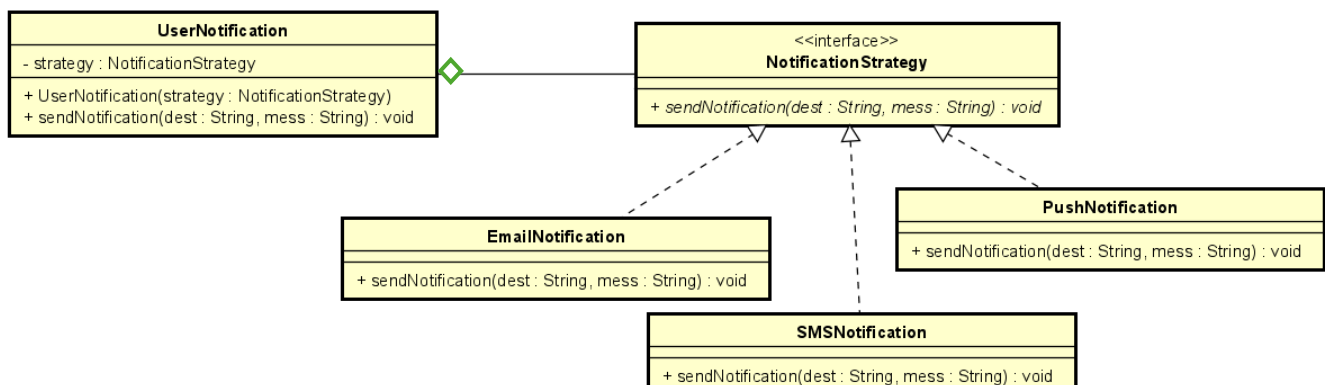


Câu (b)

Đánh giá thiết kế của lớp UserNotification:

- Mức độ coupling: Control coupling. Lý do: Biến `type` như một cờ điều khiển, quyết định hình thức gửi thông báo (qua `switch - case`).
- Mức độ cohesion: Logical cohesion. Lý do: Các phương thức `sendSMS()`, `sendEmail()`, `sendPushNotification()` có mặt ở lớp này chỉ vì cùng chức năng logic (gửi thông báo), nhưng chúng có cách xử lý chi tiết hoàn toàn khác nhau.
- Đánh giá tuân thủ SOLID:
 - Vì phạm nguyên tắc S (single responsibility): Chịu trách nhiệm xử lý logic cụ thể cho nhiều cách gửi thông báo khác nhau.
 - Vì phạm nguyên tắc O (open/close): Tương lai nếu có thêm cách gửi thông báo khác (hoặc logic của các hình thức cũ thay đổi) thì buộc phải sửa đổi lớp này.
 - Có thể vi phạm nguyên tắc D (dependency inversion): Lớp `UserNotification` phải biết logic cụ thể của các hình thức gửi thông báo (phụ thuộc trực tiếp vào cách gửi thông báo).

Cải tiến: Sử dụng strategy pattern



```

public interface NotificationStrategy {
    public void sendNotification(String dest, String mess);
}

public class EmailNotification implements NotificationStrategy {
    public void sendNotification(String dest, String mess) {
        // Logic gửi email
    }
}

public class SMSNotification implements NotificationStrategy {
    public void sendNotification(String dest, String mess) {
        // Logic gửi SMS
    }
}

public class PushNotification implements NotificationStrategy {
    public void sendNotification(String dest, String mess) {
        // Logic gửi push notification
    }
}

public class UserNotification {
    private NotificationStrategy strategy;
    public UserNotification(NotificationStrategy strategy) {
        this.strategy = strategy;
    }
    public void setStrategy(NotificationStrategy strategy) {
        this.strategy = strategy;
    }
    public void sendNotification(String dest, String mess) {
        this.strategy.sendNotification(dest, mess);
    }
}

public class Test {
    public static void main() {
        // Giả sử đã có đối tượng user và message
        // Gửi email
        UserNotification userNoti = new UserNotification(new EmailNotification());
        userNoti.sendNotification(user.getEmail(), message);
        // Chuyển sang gửi SMS
        userNoti.setStrategy(new SMSNotification());
        userNoti.sendNotification(user.getPhoneNumber(), message);
    }
}

```

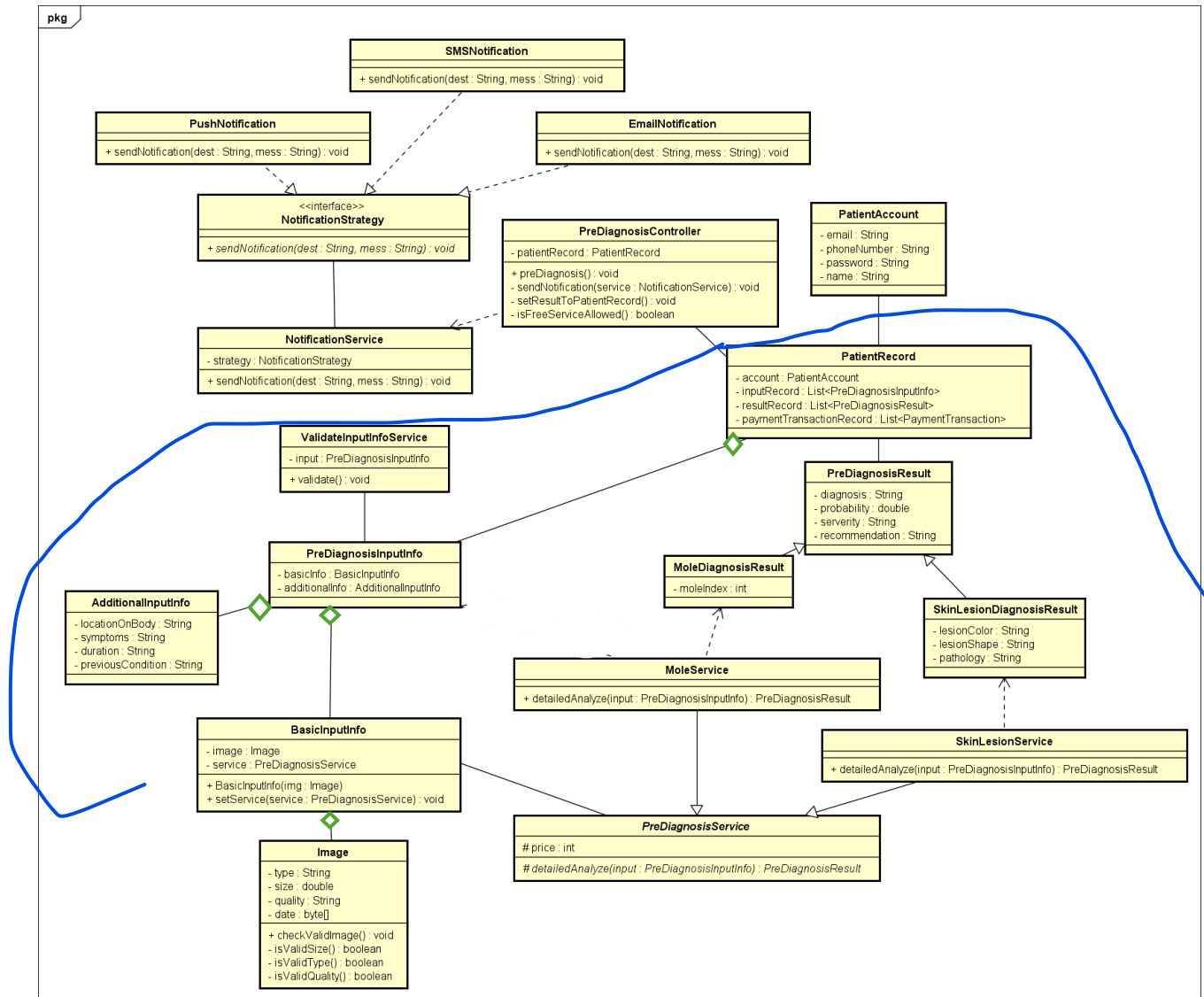
Câu (c) Có thể tham khảo bản thiết kế sau. Lưu ý:

- Thời gian có hạn, không liệt kê tất cả các phương thức trong biểu đồ lớp, chỉ cần liệt kê các phương thức thể hiện ý tưởng chính, bỏ qua constructor (*nếu không có gì đặc biệt*) và getter/setter.
- Phải vẽ biểu đồ lớp (tối đa 3.5/5 điểm). Nên vẽ nháp trước khi vẽ vào giấy thi. Thể hiện đúng mối quan hệ giữa các lớp (lưu ý: dependency, association, aggregation, composition). Ý tưởng thiết kế thể hiện ở biểu đồ lớp càng tốt thì điểm càng cao (high cohesion & low coupling, SOLID).
- Không mất thời gian viết mã nguồn (1 điểm). Mã nguồn CHỈ THỂ HIỆN Ý TƯỞNG CHÍNH. Các lớp mà chỉ có construct thuần túy và getter/setter thì KHÔNG CẦN VIẾT. Nên viết lớp Test có phương thức main để demo ý tưởng (chỉ nên viết khoảng 1-1.5 trang giấy cho mã nguồn).

- Không cần thiết kế UI và các lớp views (mất thời gian).
- Kiểm thử nên chọn kiểm thử hộp đen (phân lớp tương đương...) (0.5 điểm) để tiết kiệm thời gian.

Lưu ý: Do thời gian có hạn nên một số mối quan hệ trong biểu đồ lớp này vẽ chưa được chuẩn:

- Thuộc tính service trong BasicInputInfo nên được đưa vào PatientRecord.



```

class PreDiagnosisInputInfo {
    private BasicInputInfo basicInfo;
    private AdditionalInputInfo additionalInfo;
    public PreDiagnosisInputInfo(BasicInputInfo basicInfo, AdditionalInputInfo additionalInfo)
{ }
}

class ValidateInputService {
    public static boolean validate(PreDiagnosisInputInfo input) {
        return input.getBasicInfo().getImage().isValidSize() &&
            input.getBasicInfo().getImage().isValidType() &&
            input.getBasicInfo().getImage().isValidQuality();
        // Các logic check tính hợp lệ khác (khác null, đúng định dạng...)
    }
}

interface NotificationStrategy {
    void sendNotification(String dest, String mess);
}
  
```

```

}

class PushNotification implements NotificationStrategy {
    public void sendNotification(String dest, String mess) { // Logic gửi push noti }
}

class SMSNotification implements NotificationStrategy {
    public void sendNotification(String dest, String mess) { // Logic gửi SMS }
}

class EmailNotification implements NotificationStrategy {
    public void sendNotification(String dest, String mess) { // Logic gửi email }
}

class NotificationService {
    private NotificationStrategy strategy;
    public NotificationService(NotificationStrategy strategy) {
        this.strategy = strategy;
    }
    public void sendNotification(String dest, String mess) {
        strategy.sendNotification(dest, mess);
    }
}

abstract class PreDiagnosisService {
    protected int price;
    protected abstract PreDiagnosisResult detailedAnalyze(PreDiagnosisInputInfo input);
}

class MoleService extends PreDiagnosisService {
    public MoleService() { this.price = 15; }
    protected PreDiagnosisResult detailedAnalyze(PreDiagnosisInputInfo input) {
        return new MoleDiagnosisResult();
    }
}

class SkinLesionService extends PreDiagnosisService {
    public SkinLesionService() { this.price = 10; }
    protected PreDiagnosisResult detailedAnalyze(PreDiagnosisInputInfo input) {
        return new SkinLesionDiagnosisResult();
    }
}

class PreDiagnosisResult {
    protected String diagnosis;
    protected double probability;
    protected String severity;
    protected String recommendation;
    public PreDiagnosisResult(String diagnosis, double probability, String severity, String recommendation) { }
}

class MoleDiagnosisResult extends PreDiagnosisResult {
    private int moleIndex;
}

```

```

    public MoleDiagnosisResult(String diagnosis, double probability, String severity, String
recommendation, int moleIndex) {
        super(diagnosis, probability, severity, recommendation);
        this.moleIndex = moleIndex;
    }
}

class SkinLesionDiagnosisResult extends PreDiagnosisResult {
    private String lesionColor;
    private String lesionShape;
    private String pathology;
    public SkinLesionDiagnosisResult(String diagnosis, double probability, String severity,
String recommendation, String lesionColor, String lesionShape, String pathology) {
        super(diagnosis, probability, severity, recommendation);
        this.lesionColor = lesionColor;
        this.lesionShape = lesionShape;
        this.pathology = pathology;
    }
}

class PatientRecord {
    private PatientAccount account;
    private List<PreDiagnosisInputInfo> inputRecords = new ArrayList<>();
    private List<PreDiagnosisResult> resultRecords = new ArrayList<>();
    private List<PaymentTransaction> paymentTransactionRecords = new ArrayList<>();
    public PatientRecord(PatientAccount account) { }
    // Getter & Setter
}

class PreDiagnosisController {
    private PatientRecord patientRecord;
    public PreDiagnosisController(PatientRecord patientRecord) {
        this.patientRecord = patientRecord;
    }
    public void requestPreDiagnosis() {
        if (isFreeServiceAllowed()) {
            preDiagnosis();
        } else {
            // Gọi thanh toán trước
            preDiagnosis();
        }
    }
    public void preDiagnosis() {
        PreDiagnosisService service = getInputService();
        PreDiagnosisResult result = service.detailedAnalyze(); // Truyền input
        // Lưu kết quả & gửi thông báo
    }
    public void sendNotification() {
        NotificationService service = new NotificationService(SMSNotification\(\));
        String phoneNumber = patientRecord.getAccount().getPhoneNumber();
        String message // Kết quả khám bệnh
        service.sendNotification(phoneNumber, message);
    }
    public PreDiagnosisInputInfo getDiagnosisInput() {
        // Logic lấy thông tin đầu vào của bệnh nhân từ patientRecord (getter đơn thuần)
    }
}

```



```

    }
    public PreDiagnosisService getInputService() {
        // Logic lấy dịch vụ mà bệnh nhân chọn từ patientRecord (getter đơn thuần)
    }
    public void setResultToPatientRecord(PreDiagnosisResult result) {
        patientRecord.getResultRecords().add(result);
    }
    public boolean isFreeServiceAllowed() {
        if (patientRecord.getInputRecords().size() > 2) return false;
    }
}

public class Test {
    public static void main(String[] args) {
        // Cho bệnh nhân nhập các thông tin đầu vào
        // Check tính hợp lệ qua ValidateInputService
        PreDiagnosisInputInfo input = new PreDiagnosisInputInfo(basicInfo, addInfo);
        PatientRecord record = new PatientRecord();
        record.setInputRecords(input);
        PreDiagnosisController controller = new PreDiagnosisController(record);
        controller.requestPreDiagnosis();
    }
}

```

Kiểm thử: Chọn phương pháp kiểm thử hộp đen và kỹ thuật phân lớp tương đương để test, ví dụ:

- Các trường hợp với hình ảnh:
 - $\langle \text{size} > 2\text{MB}, \text{type} \neq \text{"jpg"}, \text{quality} == \text{"bad"} \rangle \rightarrow \text{false}$.
 - $\langle \text{size} < 2\text{MB}, \text{type} \neq \text{"png"}, \text{quality} == \text{"good"} \rangle \rightarrow \text{false}$.
 - $\langle \text{size} > 2\text{MB}, \text{type} \neq \text{"png"}, \text{quality} == \text{"bad"} \rangle \rightarrow \text{false}$.
 - $\langle \text{size} = 2\text{MB}, \text{type} == \text{"jpg"}, \text{quality} == \text{"good"} \rangle \rightarrow \text{false}$.
 - $\langle \text{size} < 2\text{MB}, \text{type} == \text{"png"}, \text{quality} == \text{"good"} \rangle \rightarrow \text{true}$.
 - ...
- Hết số lần miễn phí:
 - Kích thước inputRecords trong patientRecord $> 2 \rightarrow \text{false}$.

Câu (d): Đánh giá thiết kế hiện tại

- Tương lai nếu thêm hình thức thông báo mới: Chỉ cần viết lớp mới implements NotificationStrategy và ghi đè phương thức sendNotification() $\rightarrow$ không có vấn đề, các lớp sử dụng nó không bị ảnh hưởng (vì đã dùng strategy pattern).
- Có thể thêm loại dịch vụ chẩn đoán: vết chàm, phát ban...: Chỉ cần viết lớp mới kế thừa PreDiagnosisService, cập nhật giá của dịch vụ (ở thuộc tính price) và ghi đè phương thức detailedAnalyze() cho cách thức chẩn đoán chi tiết $\rightarrow$ không có vấn đề. Nếu dịch vụ đó có thêm các kết quả trả về, chỉ cần viết thêm lớp mới kế thừa lớp PreDiagnosisResult và thêm các thuộc tính mới là các kết quả trả về của riêng dịch vụ đó $\rightarrow$ không có vấn đề.
- Thông tin đầu vào thay đổi: Đã có lớp AdditionalInputInfo chuyên chịu trách nhiệm cho các thông tin bổ sung $\rightarrow$ Chỉ cần thay đổi lớp này, các lớp liên quan sử dụng nó không nhất thiết phải thay đổi (trừ khi muốn lấy thông tin cụ thể thì sẽ phải thay đổi).
- Nhược điểm hiện tại: Mối quan hệ giữa các lớp bị lồng nhau quá nhiều (PatientRecord $\rightarrow$ PreDiagnosisInput $\rightarrow$ BasicInputInfo $\rightarrow$ PreDiagnosisService) $\rightarrow$ Cải thiện: Có thể đưa một trong số đó trực tiếp trở thành thuộc tính của lớp PatientRecord.